

Otimizações de Caches

- **Tempo efetivo de acesso à memória:**

$$t_{\text{acesso mem}_{\text{efetivo}}} = \text{tempo}_{\text{acerto}} + \text{taxa falhas} \times \text{miss penalty}$$

- **Otimizações:** amplamente utilizadas

- **Reduzir tempo de acerto** (hit time):

- Cache L1 pequena e simples

- **Reduzir taxa de falhas:**

- Aumentar tamanho do bloco
 - Aumentar tamanho (n^o de blocos) da cache
 - Aumentar associatividade
 - Otimizações de código (pelo compilador)

- **Reduzir tempo de tratamento de falha** (miss penalty):

- Ter caches multiníveis

- **Reduzir taxa de falhas ou tempo de tratamento de falha:**

- Prefetch por hardware
 - Prefetch por software (compilador)

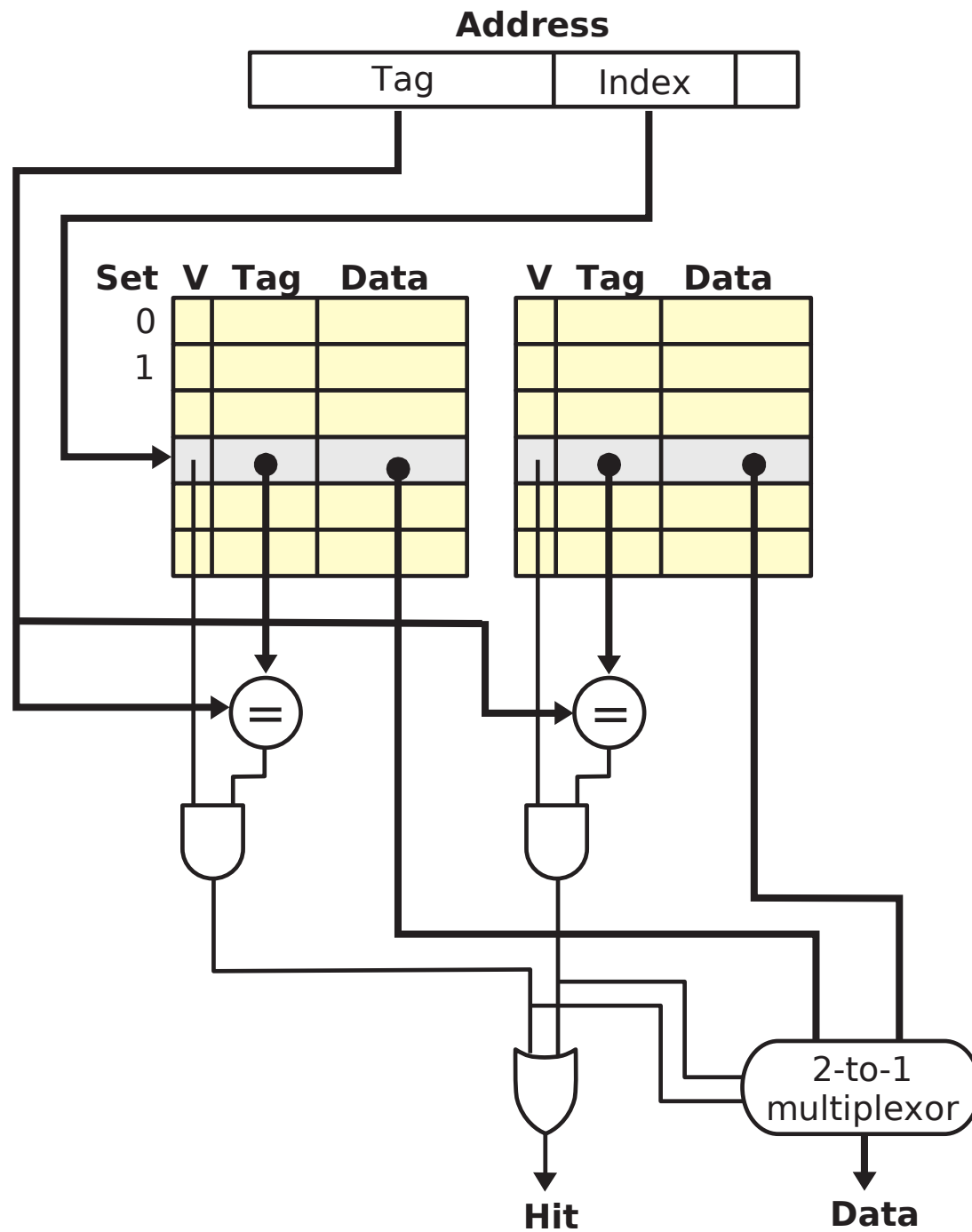
- **Reduzir tempo em stall (pela memória) do processador:**

- Usar write buffer

Redução do Tempo de Acerto: Cache Pequena e Simples

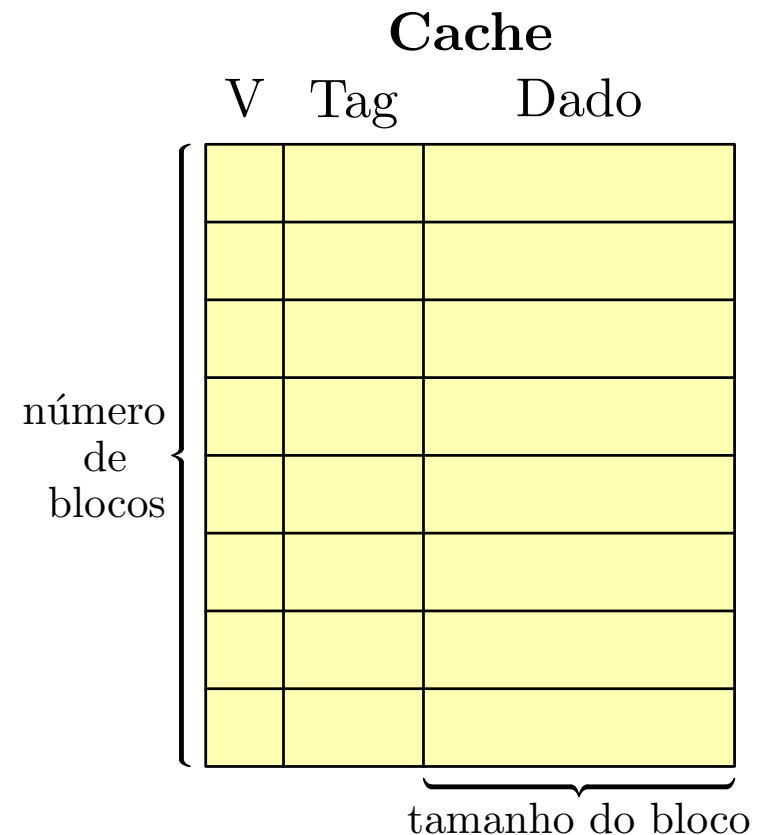
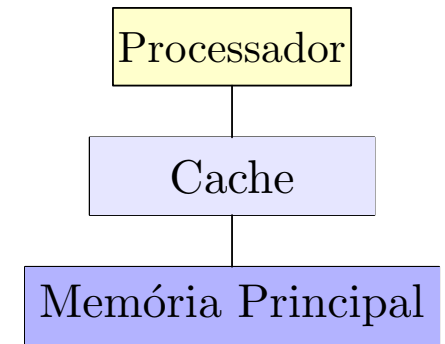
- **Tempo de acerto:**
 - Engloba:
 - Indexação do conjunto
 - Comparação de tags
 - Controle do multiplexador que seleciona entre vias do conjunto
 - Afeta tempo de ciclo do clock do processador na cache L1
- **Ideia:**
 - Usar cache com poucos blocos e associatividade pequena
- **Vantagens:**
 - Reduz tempo de acerto
 -
- **Desvantagem:**

Exemplo 1: Cache 2-way Set Associativa



Redução da Taxa de Falhas

- **Aumentar tamanho do bloco:**
 - Explora localidade espacial
 - **Vantagem:**
 - Reduz falhas _____
 - **Desvantagens?**
- **Aumentar tamanho (n^o de blocos) da cache:**
 - **Vantagem:**
 - Reduz falhas _____
 - **Desvantagens?**
- **Aumentar associatividade:**
 - Flexibiliza colocação de blocos na cache
 - **Vantagens:**
 - Reduz falhas _____
 - **Desvantagens?**



Redução da Taxa de Falhas: Otimizações de Código

- Transformações no programa realizadas pelo compilador
- **Ideia:**
 - Transformar programa para aumentar localidade temporal e espacial, nos acessos a código e/ou dados $\Rightarrow$
Reduzir taxa de falhas
- **Exemplos de otimizações:**
 - *Loop interchange*
 - *Blocking* ou *tiling*

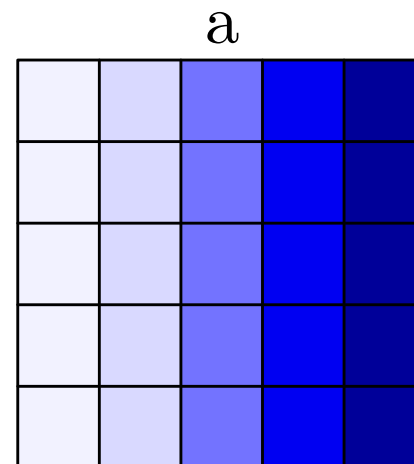
Otimizações de Código: *Loop Interchange*

- **Ideia:**
 - Trocar ordem dos laços aninhados para aumentar localidade espacial nos acessos aos elementos das estruturas de dados

- **Exemplo 2:**

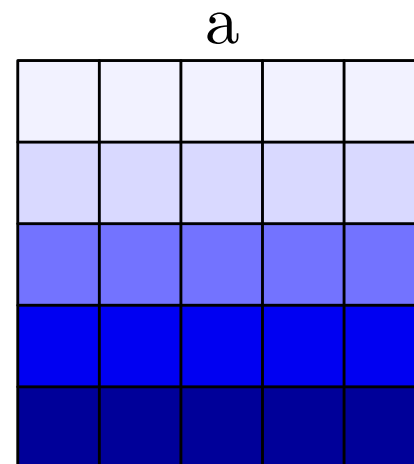
- **Antes:**

```
for (j = 0 ; j < n ; j ++)  
    for (i = 0 ; i < m ; i ++)  
        a[i][j] = 2 * a[i][j] ;
```



- **Depois:**

```
for (i = 0 ; i < m ; i ++)  
    for (j = 0 ; j < n ; j ++)  
        a[i][j] = 2 * a[i][j] ;
```



- **Vantagens?**

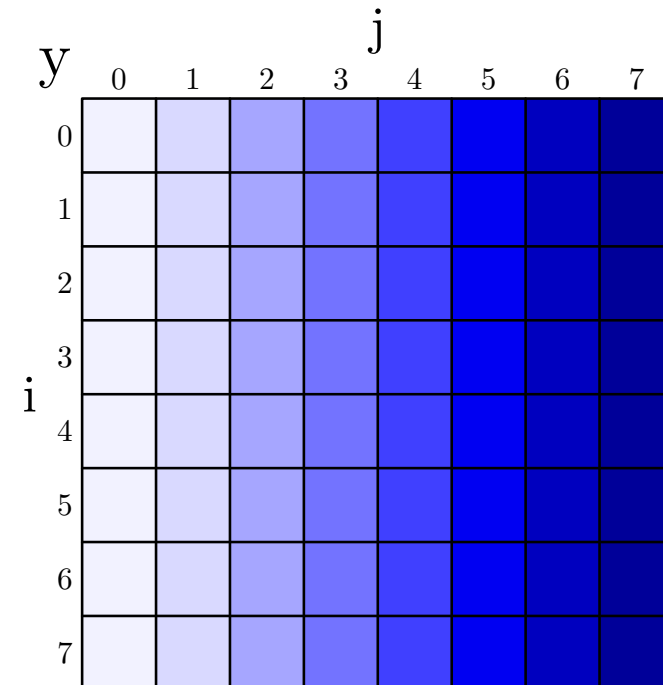
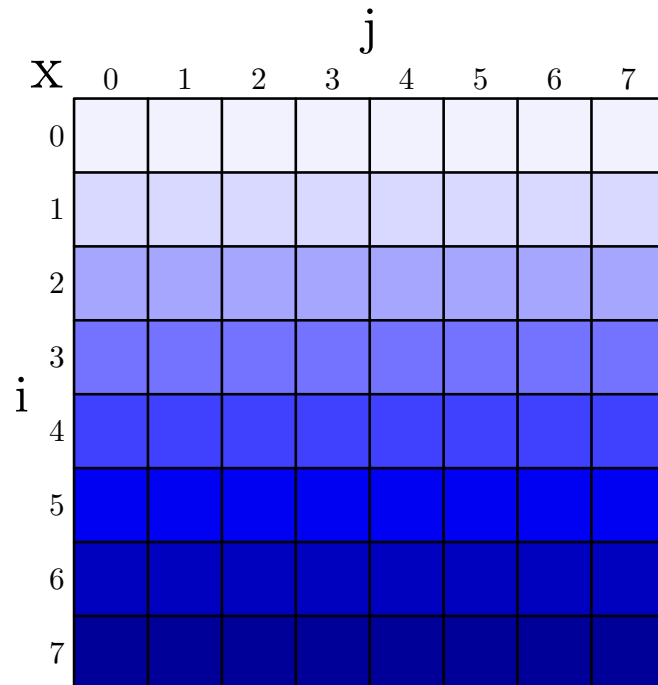
Otimizações de Código: *Blocking* ou *Tiling*

- **Ideia:**
 - Ao invés de operar linhas ou colunas inteiras, operar submatrizes (blocos), para aumentar localidade temporal e/ou espacial nos acessos aos elementos

- **Exemplo 3:** Transposição de matriz

- **Antes:**

```
for (i = 0 ; i < N ; i ++)  
    for (j = 0 ; j < N ; j ++)  
        y[j][i] = x[i][j] ;
```



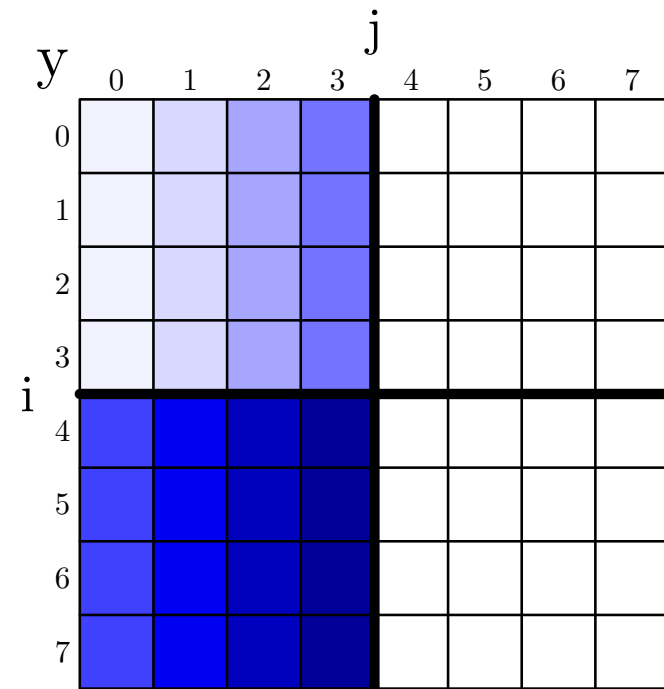
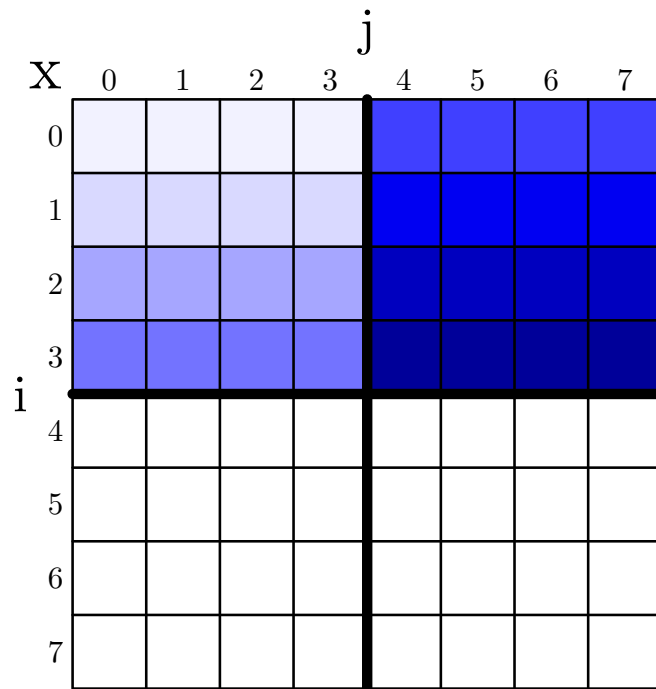
Otimizações de Código: *Blocking* (continuação)

- **Exemplo 3:** Transposição de matriz com *blocking factor* B

- **Depois:**

```
for (ii = 0; ii < N; ii = ii + B)
    for (jj = 0; jj < N; jj = jj + B)
        for (i = ii ; i < min(ii + B, N) ; i++)
            for (j = jj ; j < min(jj + B, N) ; j++)
                y[j][i] = x[i][j] ;
```

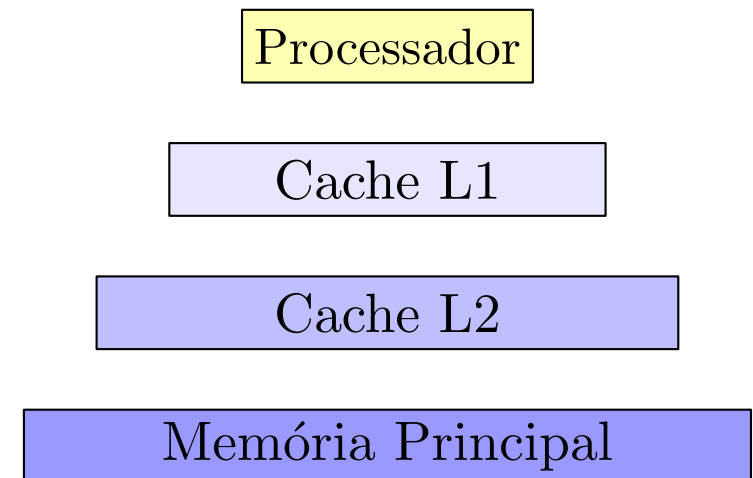
B = 4



- **Vantagem:** Reduz taxa de falhas
- **Desvantagem?**

Redução do Tempo de Tratamento de Falha: Caches Multiníveis

- **Ideia:** Ter memórias cache em mais de um nível da hierarquia de memórias
- **Cache em 2 níveis:** (Level – L)
 - **Cache L1:**
 - Mais próxima do processador (no mesmo chip)
 - Menos blocos $\Rightarrow$ Taxa de falhas maior
 - Tempo de acerto mais curto $\Rightarrow$ Afeta ciclo do clock do processador
 - Miss penalty reduzido (pela cache L2)
 - **Cache L2:**
 - Menos próxima do processador
 - Mais blocos $\Rightarrow$ Taxa de falhas menor
 - Tempo de acerto mais longo
 - Miss penalty longo
- **Acesso:**
 - Processador requisita palavra $\Rightarrow$ Busca na L1
 - Falha na L1 $\Rightarrow$ Busca na L2
 - Falha na L2 $\Rightarrow$ Busca na MP
- **Vantagem?**
- **Desvantagens?**



Organização de Caches Multiníveis

- **Split × Unified caches:**

- **Unified cache:**

- Cache unificada de instruções e dados
- Usado para L3
- **Vantagem?**

- **Split caches:**

- Cache de instruções e cache de dados separadas
- Usado para L1
- **Vantagem?**

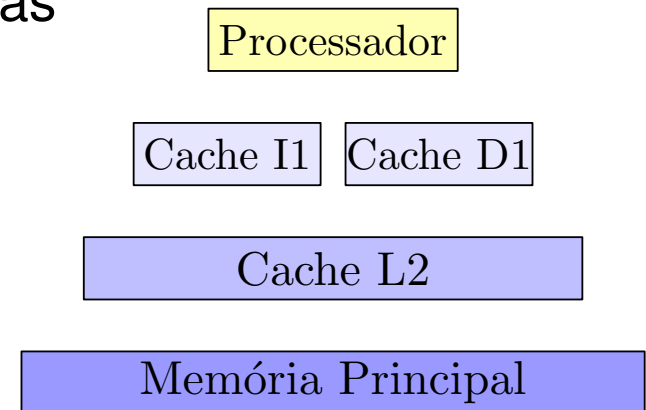
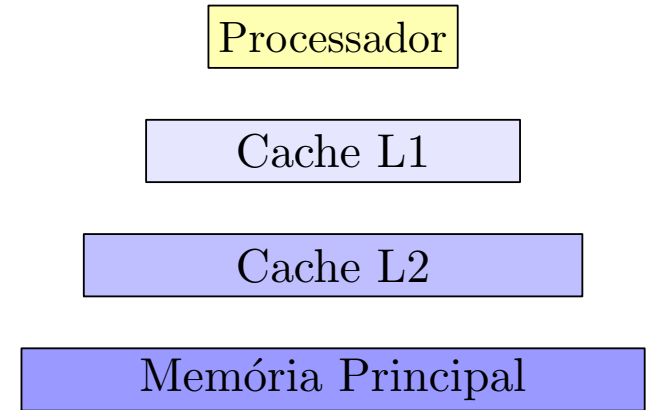
- **Entre 2 níveis de caches:**

- **Multilevel inclusion:**

- L1 contém um subconjunto dos dados da L2
- Substituição na L2 pode invalidar blocos da L1

- **Multilevel exclusion:**

- Dados na L1 não estão na L2
- Falha da L1 causa troca de blocos entre L1 e L2



Desempenho de Caches em 2 Níveis

- **Taxa de falhas L1:**

$$taxa\ falhas_{L1} = \frac{n^{\circ} falhas_{L1}}{n^{\circ} acessos_{mem}}$$

Processador

Cache L1

Cache L2

Memória Principal

- **Taxa de falhas L2:**

- **Taxa de falhas local:**

$$taxa\ falhas_{L2}^{local} = \frac{n^{\circ} falhas_{L2}}{n^{\circ} acessos_{mem} \text{ a cache } L2} = \frac{n^{\circ} falhas_{L2}}{n^{\circ} falhas_{L1}}$$

- **Taxa de falhas global:**

$$\begin{aligned} taxa\ falhas_{L2}^{global} &= \frac{n^{\circ} falhas_{L2}}{n^{\circ} acessos_{mem}} \\ &= taxa\ falhas_{L1} \times taxa\ falhas_{L2}^{local} \end{aligned}$$

Desempenho de Caches em 2 Níveis (continuação)

- Tempo efetivo de acesso à memória:

$$t_{\text{acesso mem}_{\text{efetivo}}} = t_{\text{acerto } L1} + \text{taxa falhas}_{L1} \times \text{miss penalty}_{L1}$$

$$\text{miss penalty}_{L1} = t_{\text{acerto } L2} + \text{taxa falhas}_{L2}^{\text{local}} \times \text{miss penalty}_{L2}$$

$$\begin{aligned} t_{\text{acesso mem}_{\text{efetivo}}} &= t_{\text{acerto } L1} + \text{taxa falhas}_{L1} \times (t_{\text{acerto } L2} + \text{taxa falhas}_{L2}^{\text{local}} \times \text{miss penalty}_{L2}) \\ &= t_{\text{acerto } L1} + \text{taxa falhas}_{L1} \times t_{\text{acerto } L2} + \text{taxa falhas}_{L2}^{\text{global}} \times \text{miss penalty}_{L2} \end{aligned}$$

- Ciclos em stall pela memória por instrução:

$$\begin{aligned} \text{ciclos}_{\text{stall mem por instrução}} &= \\ &\text{falhas por instrução}_{L1} \times \text{tempo acerto}_{L2} \text{ em ciclos} + \\ &\text{falhas por instrução}_{L2} \times \text{miss penalty}_{L2} \text{ em ciclos} \end{aligned}$$

Processador

Cache L1

Cache L2

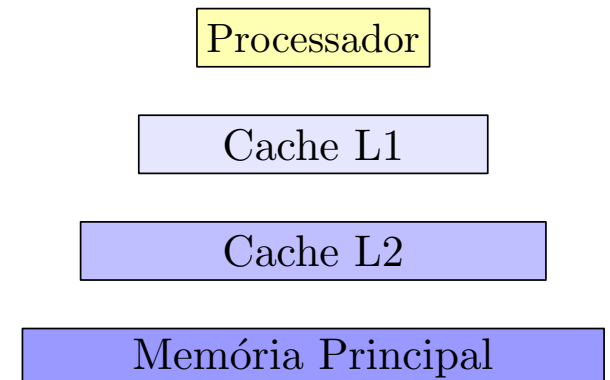
Memória Principal

Exemplo 4: Caches Multiníveis

- Processador:
 - CPI ideal = 1 (sem falhas)
- Falhas por instrução na cache L1 = 0,02
- Falhas por instrução na cache L2 = 0,005
- Tempo de acerto na cache L2 = 25 ciclos
- Tempo de acesso à MP (incluindo tratamento da falha) = 500 ciclos
- **Processador com cache L1 apenas:**
 - Ciclos em stall pela memória por instrução = _____
 - CPI real = _____
- **Processador com caches L1 e L2:**
 - Ciclos em stall pela memória por instrução = _____
 - CPI real = _____

Exemplo 5: Caches Multiníveis

- Computador com caches L1 e L2
- **Execução de um programa:**
 - N^o de instruções executadas = 800
 - N^o de acessos à memória = 1000
 - N^o de falhas na cache L1 = 40
 - N^o de falhas na cache L2 = 20
- Tempo de acerto em L1 = 1 ciclo
- Tempo de acerto em L2 = 10 ciclos
- Tempo de tratamento da falha em L2 = 100 ciclos



Exemplo 5: Caches Multiníveis (continuação)

- **Taxa de falhas:**

- Taxa de falhas L1 = _____
- Taxa de falhas global L2 = _____
- Taxa de falhas local L2 = _____

- **Tempo efetivo de acesso memória** = _____

- **Acessos à memória por instrução** = _____

- **Falhas por instrução:**

- Falhas por instrução L1 = _____
- Falhas por instrução L2 = _____

- **Ciclos em stall pela memória por instrução** = _____

Prefetch por Hardware

- **Busca blocos antecipadamente na MP:**
 - Antes de serem requisitados pelo processador
 - Blocos colocados: na **cache** ou em **buffer de prefetch**
- **Vantagem:**
 - Explora localidade espacial
 - Reduz taxa de falhas ou tempo de tratamento da falha
- **Prefetch:** de **instruções** ou **dados**
- **Exemplo 6:**
 - Com buffer de prefetch
 - Quando ocorre uma falha:
 - Se bloco solicitado **ESTÁ** no buffer de prefetch:
 - Lê bloco solicitado do buffer e coloca na cache
 - Senão:
 - Lê bloco solicitado da MP e coloca na cache
 - Lê da MP, bloco seguinte, e coloca no buffer

Memória

⋮

Cache

V	Tag	Dado

Prefetch por Software (controlado pelo Compilador)

- Usado para **prefetch de dados**
- Hardware oferece instrução de máquina de prefetch
- Compilador insere instruções de prefetch no programa, requisitando leitura de um dado antes dele ser necessário
- Enquanto dado é lido, processador continua executando
- **2 tipos de instrução de prefetch:**
 - **Prefetch para registrador:**
 - Dado lido da MP é colocado em um registrador do processador
 - **Prefetch para cache:**
 - Dado lido da MP é colocado na cache
- **Vantagem:**
 - Sobreposição execução do processador com leitura de dados na MP
 - Reduz taxa de falhas ou tempo de tratamento da falha
- **Desvantagem?**

Exemplo 7: Prefetch por Software

- **Sem prefetch:**

```
for (i = 0 ; i < n ; i ++)  
    b = b * a[i] ;
```

⇒

```
loop: LD      R1, 0 (R2)  
      DMUL    R3, R3, R1  
      DADDI   R2, R2, 4  
      ...  
      BNE     R2, R4, loop
```

- **Com prefetch para cache:**

```
loop: LD      R1, 0 (R2)    # Lê a[i] => acessa cache  
      PREFETCH 4 (R2)      # Prefetch a[i + 1] para cache  
      DMUL    R3, R3, R1  
      DADDI   R2, R2, 4  
      ...  
      BNE     R2, R4, loop
```

Redução do Tempo em Stall do Processador: Usar Write Buffer

- **Write buffer**: buffer em hardware para operações de escrita na MP
- **Ideia**:
 - Quando uma escrita na MP é necessária, ao invés de escrever diretamente na MP, escreve palavra no write buffer
 - Processador volta a executar, enquanto escrita na MP é feita a partir do buffer
 - Especialmente útil para cache com **write-through**
 - Também útil para cache com **write-back**, na substituição
- **Vantagem**:
 - Reduzir tempo em stall do processador, sobrepondo execução no processador com escrita na MP
- **Desvantagem?**

- **Exemplo 8:**

SD R1, 512 (R0) # acerto

DADD R2, R3, R4

Write Buffer

endereço	dado

Cache

V	Tag	Dado
1	T ₅₁₂	20

Memória

512 20
⋮